SocialInt

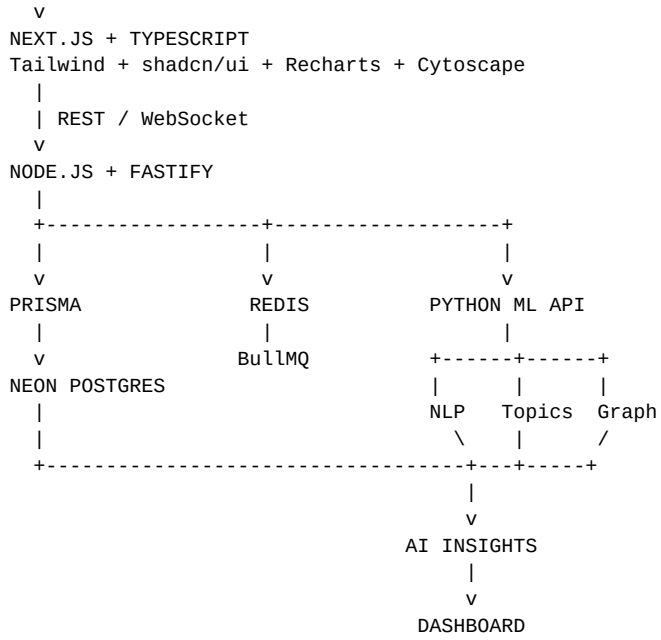
AI-Powered Multi-Platform Audience Intelligence — Full Technology Stack & Architecture

1. Core Technology Stack

Layer	Technology
Frontend	Next.js + TypeScript
Styling	Tailwind CSS
UI Components	shadcn/ui
Charts	Recharts
Network Visualization	Cytoscape.js
Maps	MapLibre GL JS
Backend Runtime	Node.js
Backend Framework	Fastify
API	REST + WebSocket
Validation	Zod
ORM	Prisma
Database	PostgreSQL
Database Hosting	Neon
Vector Search	pgvector
Cache	Redis
Job Queue	BullMQ
ML Language	Python
ML API	FastAPI
ML	PyTorch + scikit-learn
NLP	Hugging Face Transformers
Embeddings	sentence-transformers
NLP Utilities	spaCy
Data Processing	Pandas + NumPy
Graph Analysis	NetworkX
Containers	Docker
Node Package Manager	pnpm
JS Testing	Vitest
Python Testing	pytest
Frontend Deployment	Vercel
Backend / ML Deployment	Render
Version Control	Git + GitHub

2. Recommended Architecture

USER
|



3. Frontend Stack

Next.js + TypeScript is the main application layer. Use the App Router and organize the product around the major intelligence views.

Suggested pages: Overview Dashboard, Sentiment Intelligence, Audience Intelligence, Trend Explorer, Network Intelligence, and Timeline/Event Correlation.

UI: Tailwind CSS + shadcn/ui. **Charts:** Recharts. **Network:** Cytoscape.js. **Maps:** MapLibre GL JS.

4. Backend Stack

Node.js + TypeScript + Fastify will own the product/backend layer. It handles authentication, platform integrations, business logic, database access, queues, and communication with the Python ML service.

Suggested modules: auth, posts, sentiment, trends, audience, network, timeline, integrations, queues, and database.

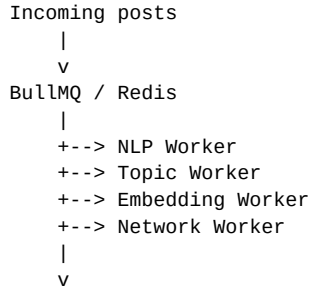
5. Database Layer

Neon PostgreSQL + Prisma is the primary persistence layer. Add **pgvector** so embeddings can be stored and searched inside PostgreSQL.

Core entities: users, posts, comments, interactions, sentiments, emotions, topics, post_topics, demographic_segments, trends, communities, and influence_scores.

6. Redis + BullMQ

Use Redis for caching, rate limiting, temporary processing state, and queues. BullMQ allows large batches of posts to be processed asynchronously rather than blocking API requests.



```
PostgreSQL / pgvector
```

7. Python AI/ML Layer

Python + FastAPI should be a separate ML service. Node.js remains the main backend, while Python owns NLP, embeddings, machine learning, and graph analytics.

Recommended libraries: PyTorch, scikit-learn, Hugging Face Transformers, sentence-transformers, spaCy, Pandas, NumPy, and NetworkX.

```
Node.js Backend
      |
      | HTTP / internal API
      v
Python FastAPI ML Service
      |
      +--> Sentiment / Emotion
      +--> Embeddings
      +--> Topic Detection
      +--> Demographic Inference
      +--> Network Metrics
```

8. AI Modules

Module	Recommended approach
Sentiment	Transformer-based classifier; compare with a classical baseline
Emotion	Multi-class / multi-label transformer classifier
Sarcasm	Optional separate classifier; add after core sentiment works
Embeddings	sentence-transformers + pgvector
Topic Detection	Embeddings + clustering such as HDBSCAN/K-Means
Trend Detection	Volume + velocity + engagement + novelty + time
Demographics	Aggregate probabilistic inference from public signals
Influence	Degree, betweenness, PageRank-style metrics
Communities	Graph community detection
Propagation	Timestamped interaction graph + community transitions

9. Data Ingestion Architecture

```
X Adapter       Telegram Adapter       Reddit/Other Adapter
     |                  |                       |
     +------------------+-----------------------+
                        |
                        v
                Common Data Schema
                        |
                        v
               Validation + Cleaning
                        |
                        v
                  PostgreSQL
                        |
                        v
                  ML Pipeline
```

Use platform adapters so the AI layer receives one normalized SocialPost / SocialUser / Interaction schema. This makes additional platforms easier to add later.

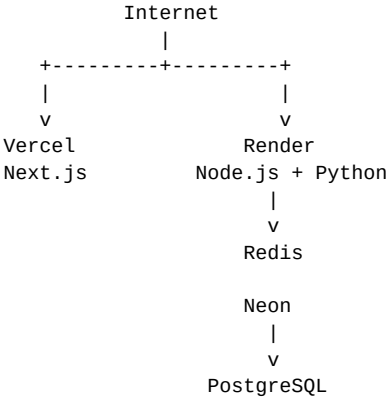
10. Suggested Repository Structure

```
socialint/
|
+-- apps/
|   +-- web/                # Next.js + TypeScript
|   +-- api/                # Node.js + Fastify
|
+-- services/
|   +-- ml/                 # Python + FastAPI
|
+-- packages/
|   +-- types/              # Shared TypeScript types
|   +-- config/
|   +-- ui/
|
+-- prisma/
|   +-- schema.prisma
|
+-- docker/
|
+-- .env
+-- package.json
+-- pnpm-workspace.yaml
+-- README.md
```

11. Real-Time Flow

```
Social platform
     |
     v
Ingestion service
     |
     v
Node.js backend
     |
     +--> Queue processing
     |
     v
PostgreSQL
     |
     v
WebSocket event
     |
     v
Next.js dashboard
     |
     v
Live charts / counters / alerts
```

12. Deployment



13. Development Strategy

Do not install/build every technology immediately. Start with the smallest useful stack and add infrastructure when the corresponding feature is needed.

Phase	Build
1. Foundation	Next.js, TypeScript, Tailwind, shadcn/ui, Node/Fastify, Prisma, Neon, GitHub
2. Data	Common schema, historical dataset, X/Telegram adapter, ingestion pipeline
3. NLP	Sentiment, emotion, language, embeddings
4. Trends	Topic extraction, clustering, trend scoring, time-series analytics
5. Network	Graph construction, centrality, communities, propagation
6. Dashboard	Overview, sentiment, audience, trends, network, timeline
7. Intelligence	AI-generated insights, event correlation, cross-platform comparison
8. Production polish	Redis, BullMQ, WebSockets, Docker, testing, deployment

14. MVP vs Advanced Scope

MVP: data ingestion + sentiment/emotion + topic/trend detection + dashboard.

Strong B.Tech version: add aggregate audience profiling, network analysis, timeline correlation, and live/replayed streaming.

SIH showcase version: combine all five core requirements with AI-generated cross-dimensional insights and an interactive influence/propagation graph.

15. Key Architectural Principle

Node.js owns the application. Python owns the intelligence. The frontend communicates with the Node API; Node handles product logic and persistence; Python provides ML predictions and graph analytics.

16. Final Stack at a Glance

```
Frontend       Next.js + TypeScript + Tailwind + shadcn/ui
Charts         Recharts
Network        Cytoscape.js
Maps           MapLibre GL JS

Backend        Node.js + Fastify + Zod
ORM            Prisma
Database       Neon PostgreSQL + pgvector
Cache/Queue    Redis + BullMQ
Realtime       WebSockets

AI/ML          Python + FastAPI
NLP            Transformers + sentence-transformers + spaCy
ML             PyTorch + scikit-learn
Data           Pandas + NumPy
Graph          NetworkX

DevOps         Docker + pnpm
Testing        Vitest + pytest
Deployment     Vercel + Render + Neon
Git            GitHub
```